

Assignment #7: bfs、

Updated 0851 GMT+8 Oct 21, 2025

2025 fall, Compiled by 窦兆文 元培学院

说明：

1. 解题与记录：

对于每一个题目，请提供其解题思路（可选），并附上使用Python或C++编写的源代码（确保已在OpenJudge，Codeforces，LeetCode等平台上获得Accepted）。请将这些信息连同显示“Accepted”的截图一起填写到下方的作业模板中。（推荐使用Typora <https://typoraio.cn> 进行编辑，当然你也可以选择Word。）无论题目是否已通过，请标明每个题目大致花费的时间。

2. **提交安排：**提交时，请首先上传PDF格式的文件，并将.md或.doc格式的文件作为附件上传至右侧的“作业评论”区。确保你的Canvas账户有一个清晰可见的本人头像，提交的文件为PDF格式，并且“作业评论”区包含上传的.md或.doc附件。

3. **延迟提交：**如果你预计无法在截止日期前提交作业，请提前告知具体原因。这有助于我们了解情况并可能为你提供适当的延期或其他帮助。

请按照上述指导认真准备和提交作业，以保证顺利完成课程要求。

1. 题目

M23555: 节省存储的矩阵乘法

implementation, matrices, <http://cs101.openjudge.cn/practice/23555>

要求用节省内存的方式实现，不能还原矩阵的方式实现。

思路：

由于使用稀疏矩阵表示（只存储非零元素），我们需要：

- 1.重组数据结构：将矩阵X按行索引组织，矩阵Y按列索引组织，便于快速查找
- 2.计算乘积：只有当 $X[i][k]$ 和 $Y[k][j]$ 都非零时，才会对 $C[i][j]$ 产生贡献
- 3.输出结果：按行列号排序输出非零元素

代码：

```

n, m1, m2 = map(int, input().split())
row_X = [[] for _ in range(n)]
for _ in range(m1):
    i, j, val = map(int, input().split())
    row_X[i].append((j, val))
col_Y = [[] for _ in range(n)]
for _ in range(m2):
    i, j, val = map(int, input().split())
    col_Y[j].append((i, val))
result = {}

for i in range(n):
    if not row_X[i]:
        continue

    for j in range(n):
        if not col_Y[j]:
            continue
        sum_val = 0
        x_dict = dict(row_X[i])

        for k, y_val in col_Y[j]:
            if k in x_dict:
                sum_val += x_dict[k] * y_val
        if sum_val != 0:
            result[(i, j)] = sum_val
sorted_result = sorted(result.items())
for (i, j), val in sorted_result:
    print(i, j, val)

```

代码运行截图（至少包含有"Accepted"）

#50588487提交状态

状态: **Accepted**

M102.二叉树的层序遍历

bfs, <https://leetcode.cn/problems/binary-tree-level-order-traversal/>

思路：

BFS（广度优先搜索）- 使用队列

代码：

```
class Solution(object):
    def levelOrder(self, root):
        if not root:
            return []

        result = []
        queue = [root]

        while queue:
            level_size = len(queue)
            current_level = []

            for _ in range(level_size):
                node = queue.pop(0)
                current_level.append(node.val)

                if node.left:
                    queue.append(node.left)
                if node.right:
                    queue.append(node.right)

            result.append(current_level)

        return result
```

代码运行截图（至少包含有"Accepted"）

通过 35 / 35 个通过的测试用例

 Youthful Spencezwr 提交于 2025.10.27 17:56

 官方题解

 写题解



字节跳动心动计划

面向短期冲刺字节跳动选手，求职弯道顺利超车



🕒 执行用时分布



0 ms | 击败 100.00% 🏆

🌟 复杂度分析

💾 消耗内存分布

12.89 MB | 击败 87.14% 🏆

🌟 复杂度分析

M131.分割回文串

dp, backtracking, <https://leetcode.cn/problems/palindrome-partitioning/>

思路：

回溯 + 动态规划预处理

代码：

```

class Solution(object):
    def partition(self, s):
        n = len(s)
        result = []
        path = []
        is_palindrome = [[False] * n for _ in range(n)]

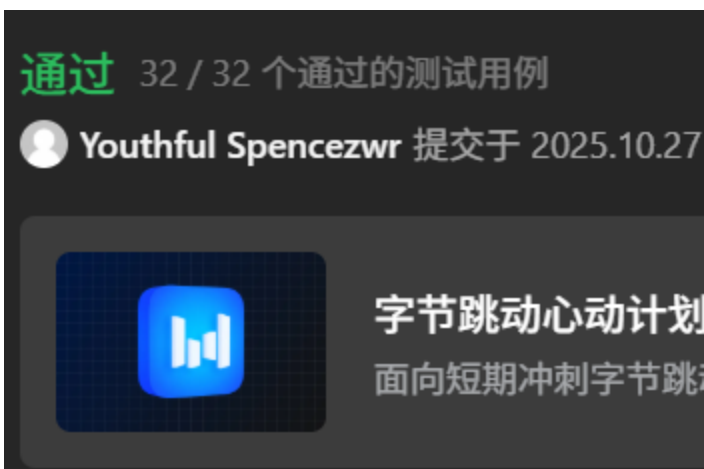
        for j in range(n):
            for i in range(j + 1):
                if s[i] == s[j] and (j - i <= 2 or is_palindrome[i + 1][j - 1]):
                    is_palindrome[i][j] = True

        def backtrack(start):
            if start == n:
                result.append(path[:])
                return
            for end in range(start, n):
                if is_palindrome[start][end]:
                    path.append(s[start:end + 1])
                    backtrack(end + 1)
                    path.pop()

        backtrack(0)
        return result

```

代码运行截图（至少包含有"Accepted"）



M200.岛屿数量

dfs, bfs, <https://leetcode.cn/problems/number-of-islands/>

思路：

DFS（深度优先搜索）

代码

```
class Solution(object):
    def numIslands(self, grid):
        if not grid or not grid[0]:
            return 0

        rows, cols = len(grid), len(grid[0])
        count = 0

        def dfs(r, c):
            if r < 0 or r >= rows or c < 0 or c >= cols or grid[r][c] == '0':
                return
            grid[r][c] = '0'
            dfs(r + 1, c)
            dfs(r - 1, c)
            dfs(r, c + 1)
            dfs(r, c - 1)

        for r in range(rows):
            for c in range(cols):
                if grid[r][c] == '1':
                    count += 1
                    dfs(r, c)

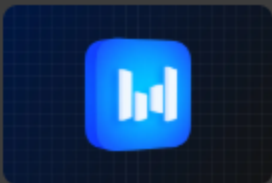
        return count
```

(至少包含有"Accepted")

通过 49 / 49 个通过的测试用例



Youthful Spencezwr 提交于 2025.10.2



字节跳动心动计
面向短期冲刺字节

1123.最深叶节点的最近公共祖先

dfs, <https://leetcode.cn/problems/lowest-common-ancestor-of-deepest-leaves/>

思路:

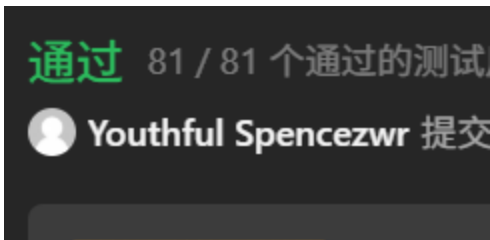
DFS - 返回深度和节点

代码

```
class Solution(object):
    def lcaDeepestLeaves(self, root):
        def dfs(node):
            if not node:
                return (0, None)
            left_depth, left_lca = dfs(node.left)
            right_depth, right_lca = dfs(node.right)
            if left_depth > right_depth:
                return (left_depth + 1, left_lca)
            elif right_depth > left_depth:
                return (right_depth + 1, right_lca)
            else:
                return (left_depth + 1, node)

        return dfs(root)[1]
```

(至少包含有"Accepted")



M79.单词搜索

回溯, <https://leetcode.cn/problems/word-search/>

思路:

DFS回溯

代码:

```

class Solution(object):
    def exist(self, board, word):
        if not board or not board[0]:
            return False

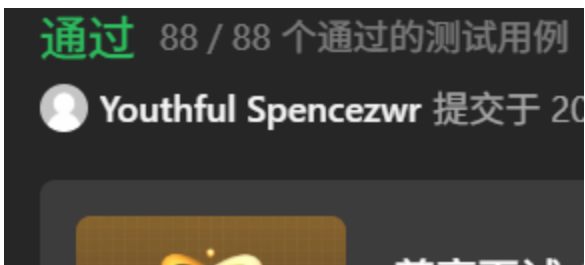
        rows, cols = len(board), len(board[0])

        def dfs(r, c, index):
            if index == len(word):
                return True
            if (r < 0 or r >= rows or c < 0 or c >= cols or
                board[r][c] != word[index]):
                return False
            temp = board[r][c]
            board[r][c] = '#'
            found = (dfs(r + 1, c, index + 1) or
                    dfs(r - 1, c, index + 1) or
                    dfs(r, c + 1, index + 1) or
                    dfs(r, c - 1, index + 1))
            board[r][c] = temp
            return found
        for r in range(rows):
            for c in range(cols):
                if board[r][c] == word[0] and dfs(r, c, 0):
                    return True

        return False

```

代码运行截图（至少包含有"Accepted"）



2. 学习总结和个人收获

如果发现作业题目相对简单，有否寻找额外的练习题目，如“数算2025fall每日选做”、LeetCode、Codeforces、洛谷等网站上的题目。

通过本周的学习，了解了树形数据结构的核心概念和算法实现,从基础的二叉树遍历、哈夫曼编码,到高级的AVL平衡树、并查集等数据结构,理解了递归思想在树操作中的重要性。